

Flag : `picoCTF{1t_w4snt_th4t_34sy_tbh_...}`

📄 Résumé de la faille

Un fichier PCAP contient ~22 paquets TCP déguisés en "retransmissions". Chaque paquet cache un fragment de données encodé en base64. Les paquets sont volontairement dans le **désordre temporel**. Il faut : extraire les fragments, les **trier par timestamp**, les assembler, convertir l'hex en texte, puis décoder le base64 → flag.

Concept : exfiltration de données cachée dans du trafic réseau + ordre temporel comme clé.

⚡ Méthodo — les étapes

- 1. Explorer le PCAP (comprendre)** - Ouvrir dans Wireshark (GUI) pour observer : trafic bizarre (toutes des "TCP Retransmission" SYN), timestamps en désordre (temps négatifs), fragments base64 dans les données (finissent par `=`). - Indices de l'énoncé : "well timely manner" (le TIMING) + "right filters".
- 2. Basculer en ligne de commande (la bonne approche)** - Le GUI sert à explorer/comprendre. Pour extraire et traiter en masse, le CLI (tshark + pipes Unix) est bien plus efficace et reproductible.

📄 Les commandes (Kali)

Télécharger le PCAP :

```
wget https://challenge-files.picoctf.net/.../myNetworkTraffic.pcap
```

Vérifier le fichier :

```
ls -la myNetworkTraffic.pcap
```

Extraire les données TCP (test) :

```
tshark -r myNetworkTraffic.pcap -T fields -e tcp.segment_data
```

→ sort les fragments hex, dans l'ordre du fichier (désordonné).

Extraire temps + données (pour trier) :

```
tshark -r myNetworkTraffic.pcap -T fields -e frame.time_epoch -e tcp.segment_data
```

→ deux colonnes : timestamp [tab] fragment_hex.

LA commande finale :

```
tshark -r myNetworkTraffic.pcap -T fields -e frame.time_epoch -e tcp.segment_data | sort | cut -f2 | tr -d '\n' | xxd -r -p | base64 -d
```

→ flag `picoCTF{...}`

🔪 Décorticage de la commande finale

Maillon	Rôle
<code>tshark -r f -T fields -e ...</code>	Extrait timestamp + données TCP du pcap
<code>\ sort</code>	Trie les lignes par timestamp (remet dans l'ordre)
<code>\ cut -f2</code>	Garde seulement la colonne 2 (le fragment hex)
<code>\ tr -d '\n'</code>	Supprime les retours à la ligne (colle tout)
<code>\ xxd -r -p</code>	Convertit l'hex en texte (= le base64)
<code>\ base64 -d</code>	Décode le base64 → le flag

Le `|` (pipe) envoie la sortie d'une commande vers la suivante. Philosophie Unix : chaque outil fait UNE chose, on les enchaîne.

Schéma : extraire → trier → garder les data → coller → hex vers texte → décoder

📄 tshark — les bases (Wireshark en CLI)

```
tshark -r fichier.pcap -T fields -e CHAMP
```

Option	Rôle
-r fichier	Lire un fichier de capture (read)
-T fields	Sortie en champs bruts (pour extraire des valeurs)
-e champ	Quel champ extraire (répérable : plusieurs -e)
-Y "filtre"	Appliquer un filtre d'affichage (ex: -Y "http.request")
-c N	Limiter à N paquets

Champs utiles (mêmes noms que les filtres Wireshark) : | Champ | Contenu | |---|---| | frame.time_epoch | Timestamp (secondes depuis 1970) | | frame.number | Numéro du paquet | | ip.src / ip.dst | IP source / destination | | tcp.port | Port TCP | | tcp.segment_data | Données TCP (payload, en hex) | | http.request.uri | URLs demandées | | dns.qry.name | Domaines demandés |

📦 Commandes Unix apprises (réutilisables partout)

Commande	Rôle
wget [url]	Télécharger un fichier (simple, enregistre direct)
curl -O [url]	Idem, mais curl sert surtout à INTERAGIR (API, POST, headers)
ls -la	Lister fichiers avec détails
sort	Trier les lignes (-n pour tri numérique)
cut -f2	Garder la colonne 2 (séparateur = tabulation)
tr -d '\n'	Supprimer des caractères (ici les retours ligne)
xxd -r -p	Hex → texte (-r reverse, -p plain)
base64 -d	Décoder du base64

wget vs curl : wget pour RÉCUPÉRER un fichier, curl pour INTERAGIR avec un serveur (forger des requêtes, tester des API, headers custom).

✔️ Réflexes à retenir

- **GUI (Wireshark) pour explorer/comprendre, CLI (tshark) pour extraire/traiter.**
- Fragments qui finissent par `=` → base64.
- Données en hex → `xxd -r -p` pour convertir en texte.
- Timestamps en désordre + indice "timing" → il faut TRIER par le temps.
- Chaîner les outils avec des pipes `|` : extraire → trier → nettoyer → décoder.
- Pas besoin de retenir les commandes par cœur : comprendre la logique, garder les commandes dans ses notes.